The Journal of Open Source Education

scicode-widgets: Bringing Computational Experiments to the Classroom with Jupyter Widgets

Alexander Goscinski[1], **Taylor J. Baird**[2], **Dou Du**[2,3], **João Prado**[4], **Divya Suman**[4], **Tulga-Erdene Sodjargal**[4], **Sara Bonella**[2], **Giovanni Pizzi**[1,5], **and Michele Ceriotti**[4]

1 PSI Center for Scientific Computing, Theory and Data, 5232 Villigen PSI, Switzerland **2** Centre Européen de Calcul Atomique et Moléculaire (CECAM), École Polytechnique Fédérale de Lausanne, 1015 Lausanne, Switzerland **3** Theory and Simulation of Materials (THEOS), École Polytechnique Fédérale de Lausanne, 1015 Lausanne, Switzerland **4** Laboratory of Computational Science and Modeling, École Polytechnique Fédérale de Lausanne, 1015 Lausanne, Switzerland **5** National Centre for Computational Design and Discovery of Novel Materials (MARVEL), 5232 Villigen PSI, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Submitted: 02 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

"Computational experiments" use code and interactive visualizations to convey mathematical and physical concepts in an intuitive way, and are increasingly used to support *ex cathedra* lecturing in scientific and engineering disciplines. Jupyter notebooks are particularly well-suited to implement them, but involve large amounts of ancillary code to process data and generate illustrations, which can distract students from the core learning outcomes. For a more engaging learning experience that only exposes relevant code to students—allowing them to focus on the interplay between code, theory and physical insights—we developed `scicode-widgets` (released as scwidgets), a Python package to build Jupyter-based applications. The package facilitates the creation of interactive exercises and demonstrations for students in any discipline in science, technology and engineering. Students are asked to provide pedagogically meaningful contributions in terms of theoretical understanding, coding ability, and analytical skills. The library provides the tools to connect custom pre- and post-processing of students' code, which runs seamlessly "behind the scenes", with the ability to test and verify the solution, as well as to convert it into live interactive visualizations driven by Jupyter widgets.

Statement of Need

This work introduces scicode-widgets, an open-source Python library that transforms Jupyter notebooks into interactive, self-contained learning applications. By hiding boilerplate code behind a clean widget interface, it allows students to focus on core physics concepts, algorithms, and data analysis with instant visual feedback. The widget configuration is fully implemented in Python, enabling flexible creation of diverse teaching exercises—particularly valuable for the research community, given Python's widespread use in academic, educational, and research settings.

We demonstrate its educational value through a ridge regression exercise that combines interactive code input, real-time controls, and automated visual feedback supported by a color-coded cue system. Used in one undergraduate course at EPFL (MSE-305), the tool has already supported nearly 100 students and received highly positive feedback.

The Journal of Open Source Education

Introduction

Jupyter notebooks (Thomas et al., 2016) have been used extensively for the creation of educational contents, with applications in chemistry (Stokes, 2021), physics (Hu et al., 2021; Tufino et al., 2025; Urcelay-Olabarria et al., 2017) and mathematics (Sümmermann et al., 2021). The cell-based structure of the notebook supports conveying the teaching material in smaller units that can be connected in a sequential coherent narrative, thereby facilitating learning (Granger & Perez, 2021). In addition to bare notebooks, custom Jupyter widgets have been developed both in research contexts (Yakutovich et al., 2021) and in educational contexts to create virtual lab applications, e.g., allowing students to explore the experimental parameter space interactively (Du et al., 2023, 2024; Sutchenkov & Tikhonov, 2020) or simply serving as a replacement for a hands-on lab experience (Heradio et al., 2016). Indeed, studies in control education have observed that these learning experiences can reach a similar effectiveness as their hands-on counterparts (Heradio et al., 2016). These examples usually focus on conveying the interaction between theory and experimental results to students. However, they do not teach how to create and design these computational experiments, a skill that is becoming more essential for all fields of computational science. Although related skills are taught in separate courses, the interplay of theoretical understanding, algorithmic design, and interpretation of the results is typically only taught in more advanced courses. A crucial contributing factor to its late introduction in the curriculum is the amount of boilerplate code that needs to be written to process and visualize the data, together with the amount of time needed to get acquainted with the domain-specific tools that help with this kind of processing. While the code for data processing can be provided by the teacher, it still dilutes students' focus if the notebook is populated by ancillary code that does not contribute to the core learning outcomes. In addition, for an interactive experience, computational experiments need to be rerun frequently to assess the results under different conditions. While parameters in a notebook cell can be changed and rerun, it still adds friction to the experience of students if multiple actions are required to rerun an experiment (such as changing values in several cells and rerunning them in a specific order). Due to these aspects, learning is hindered by having to perform repetitive actions. To address these challenges, we developed the Python library `scicode-widgets` (released as `scwidgets`) that facilitates the implementation of Jupyter-based web applications, providing a seamlessly interactive learning experience of computational experiments to students. `scicode-widgets` allows teachers to expose only the essential code, parameters, and results (such as interactive plots) to students, while hiding the unnecessary details in the background, thus helping them focus first on the core scientific and computational concepts.

DRAFT

The Journal of Open Source Education

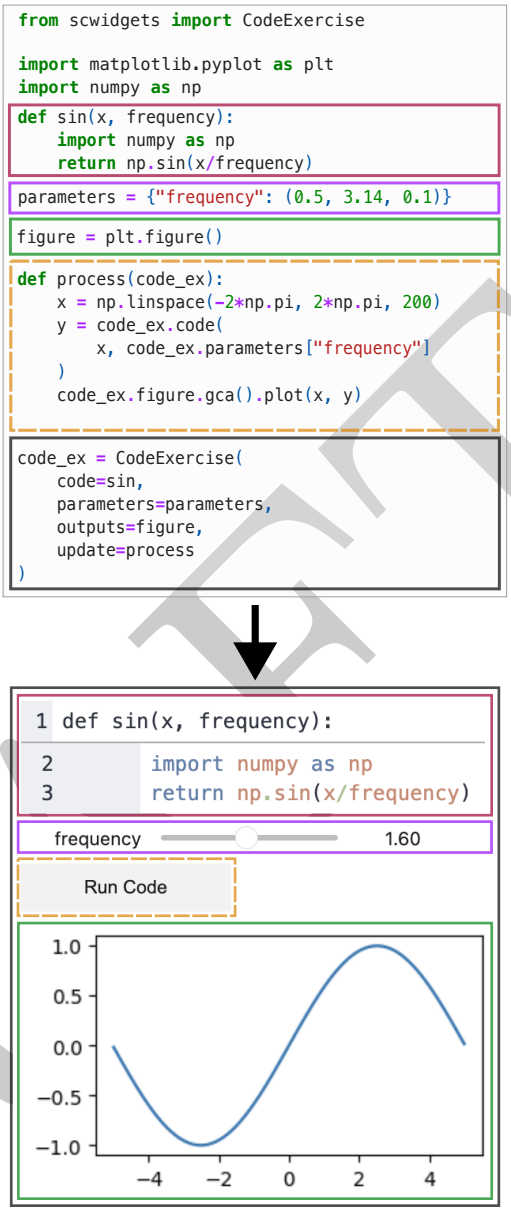

Figure 1: An example of how the coding environment, the parameter panel and the visualization of an interactive exercise widget are defined by the teacher (top) and how they are visualized by the corresponding widgets (bottom).

DRAFT

Computational Experiments

The main use case of `scicode-widgets` is the creation of flexible code exercises or demonstrations for students in the form of an interactive application built by combining several Jupyter widgets. This application can be divided into three parts that are marked with different colors in the bottom part of Fig. 1: A coding environment (red) in which students can implement the algorithm derived from the provided theory, a panel of parameters (purple) where students can manipulate the experimental settings, and a visualization (green) that is generated from the code of the students utilizing the experimental settings specified in the parameter panel. To provide full flexibility in the creation of an exercise, the teacher implements a function (orange box in the upper panel

The Journal of Open Source Education

of Fig. 1) that defines the logic to process the parameters specified in the parameters panel, and to input these as arguments to the code supplied by the students, along with functionality to visualize the results of the code execution, e.g. in Fig. 1 in the form of a plot. This process function is then executed by a click of the "Run code" button (marked in orange in the bottom panel of Fig. 1). Alternatively, the code exercise can be configured to run such process function upon the change of any of the parameters in the panel of parameters. This simple example shows how the interactive view of a notebook based on `scicode-widgets` hides away plotting and data processing code, generating a clean interface that helps the student focus on pedagogically meaningful elements.

```
check_registry = CheckRegistry()

code_ex = CodeExercise(
    code=sin,
    check_registry=check_registry,
)

check_registry.add_check(
    code_ex,
    asserts=[
        assert_type, # checks if same type
        assert_shape, # checks if same shape
        assert_numpy_allclose, # checks if allclose
    ],
    inputs_parameters=[{"x": np.asarray([0., np.pi, 2*np.pi])}],
    outputs_references=[(np.asarray([0., 0., 0.]),)]
)

def assert_2pi_periodic():
    if not np.allclose(code_ex.code(0), code_ex.code(2*np.pi)):
        return "Function is not periodic."
    return "" # empty strings means it passes

check_registry.add_check(
    code_ex,
    asserts=[
        assert_2pi_periodic,
    ]
)

check_registry.add_check(
    code_ex,
    asserts=assert_numpy_allclose,
    fingerprint = lambda x: np.sum(x),
    inputs_parameters=[{"x": np.asarray([0., np.pi, 2*np.pi])}],
    outputs_references=[(0.,),],
)
```

```
def sin(x):
    import numpy as np
    return np.cos(x) # Whoops! Wrong function
```

Check Code

```
Check failed
------------
assert_type passed for input
  x:  array([0.        , 3.14159265, 6.28318531])
assert_shape passed for input
  x:  array([0.        , 3.14159265, 6.28318531])
assert_numpy_allclose failed for
|   x: array([0.        , 3.14159265, 6.28318531])
| > output 0: [ 1. -1.  1.]
| Output is not close to reference absolute difference is 3.0,
relative difference is 3.0.
```

```
Check was successful
--------------------
assert_2pi_periodic passed
```

```
Check failed
------------
assert_numpy_allclose failed for
|   x: array([0.        , 3.14159265, 6.28318531])
```

Figure 2: Examples of the check functionality. Each check is framed with the same color as its corresponding output. The purple check verifies the built-in Python type, shape and value closeness between the output of the students' code and some reference output provided by the teacher. The blue check verifies the periodicity of the function provided by the students, i.e., $\sin(0) = \sin(2\pi)$. The yellow-green check uses a fingerprint function, demonstrating how to prevent disclosing the reference output to the students. For instance, in this example only the sum of the values of the function on a specified grid of x values (here only composed of three points for illustrative purposes) is checked against a reference value.

DRAFT

The Journal of Open Source Education

Check solutions

The instructor can also define checks for the code provided by the students, to support them with a quick feedback on the correctness of their answer, thus facilitating approaching the correct solution. An example is provided in Fig. 2. A variety of checks typically used in scientific environments are already implemented, including type and equality checks, as well as shape and numerical closeness checks for `numpy` arrays. These checks work as unit tests, validating the output of the code of the students for a set of input and reference values provided by the teacher. The instructor can also create checks that validate specific functional behavior, e.g. periodicity for trigonometric functions (for an example, see the check framed by a blue box in Fig. 2). One noteworthy point is that, for certain exercises, the reference values can contain hints of the solution, which can be exploited by students to solve the exercise. As the reference values need to be part of the notebook, they cannot be safely hidden from the students. The library therefore provides an option to pass a one-way function, analogous to a hashing or fingerprint function, that post-processes the output before the check is run. Since no inference can be made once the output is processed through a one-way function, the reference values of the checks do not reveal any information about the actual solution of the exercise. This fingerprint function depends highly on the domain and therefore needs to be provided by the instructor. Nevertheless, the library exposes a simple interface to facilitate providing such checks. An example of a check using the fingerprint functionality can be seen in Fig. 2, framed by a yellow-green box.

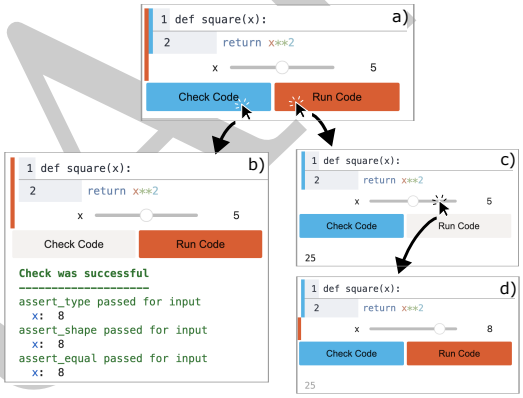

Figure 3: An example of how the cue system works, showing a sequence of widget interactions. Panel a) shows both blue and red cues active, resulting for instance from a previous change by the student of both the code and the value of the x parameter, without yet having rerun neither the code (to update the visualization) nor the checks. From a) to b) the "Check Code" button is pressed, removing the blue cues from the button and the code input (but keeping the red cues, as the code has not been rerun yet, and thus any related visualization are not yet updated). From a) to c) the "Run Code" button is pressed, removing the red cues from the code input, the parameter panel and the button (but keeping the blue cues, as the checks have not been run yet). From c) to d) the parameter x is changed, resulting in a new red cue both on the parameter panel and the button.

Cues for readability

While students work with `scicode-widgets`, they might often change their provided answer without immediately rerunning the code (to update the visualizations) or performing the related checks. This might leave the student unaware of which parts of the widget application are outdated, i.e., which parts need to be rerun to update the outputs. We therefore implemented a cue system that highlights the parts of the widget application that were modified since the last update and thus need to be rerun. When a student modifies an input in the widget, the system highlights the affected part with vertical bars of different colors – referred to as a "cues" – depending on the corresponding needed action: blue

DRAFT

The Journal of Open Source Education

for checking, and red for running the code. This indicates that the corresponding output is outdated and immediately visualizes which inputs have been changed. An example of several interactions and their effect on the cues can be seen in Fig. 3. This real-time feedback ensures that students are always aware of the current state of the outputs and can understand more easily the effect of their modifications, resulting in a more dynamic learning process.

Learning ridge regression through an interactive notebook

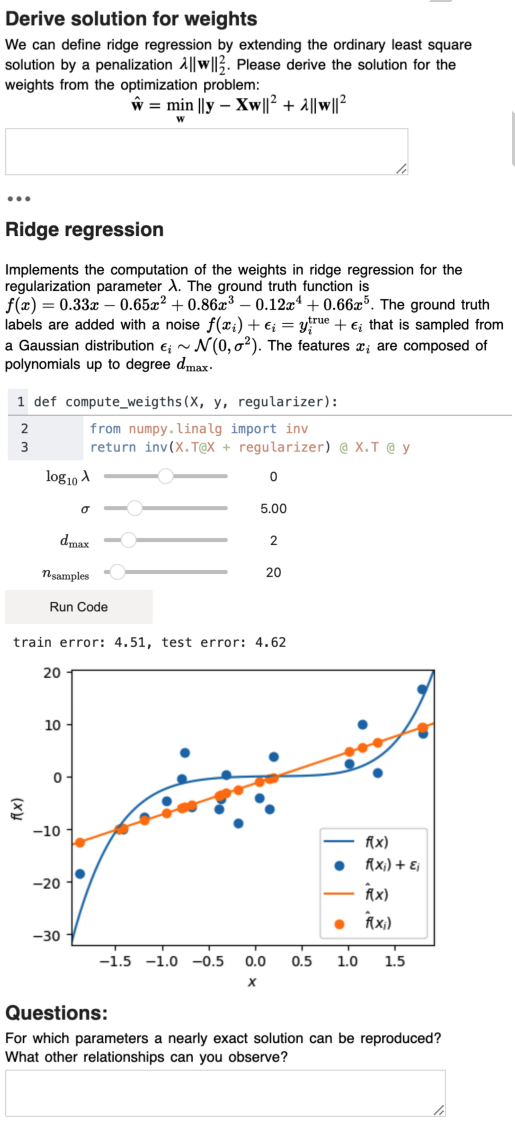

Figure 4: An example notebook covering the topic of ridge regression, divided into exercises demanding mathematical coding and analytical skills. It includes both our exercise widget and free-text boxes for answering questions. Even just generating this simple plot requires more than 30 lines of Python code. Thanks to `scicode-widgets`, this code is hidden from students, so that they can instead focus on understanding the core concepts of ridge regression and only implement the core functions (in this case, they only need to provide the body of the `compute_weights` Python function).

In this section, we showcase the use of `scicode-widgets` for an exercise aimed at explaining ridge regression (Bishop, 2006) to students. A notebook that implements such an exercise is shown in Fig. 4. In the following subsections we briefly discuss ways in which the code exercise creates a more engaging learning experience with respect to more traditional learning approaches.

Validate solutions and identify mistakes

An interesting approach to use the application is to first ask students to derive a solution for the weights from the optimization problem defining ridge regression, and then to implement it in the code environment widget. If the students make a mistake in the mathematical derivation or in the code implementation, they need to identify where the problem lies. While the instructor could provide explicit checks so that students can validate their solution, a more engaging learning experience is created by providing hints on how to use the visualization for validation. Students could then for instance reason that, when the noise and regularization strength are very small, the solution of ridge regression approaches the exact solution, which is clearly illustrated in the plot produced by the widget app. The instructor could help focus students' attention by hinting at this fact with a question of the form "For which parameters can a nearly exact solution be reproduced? Can you observe this with your code?", thereby provoking the train of thought to use this as a validation. Such an experience teaches students to find mistakes in complex processes, requiring them to systematically exclude parts by validating their correctness and identify the parts that are relevant for investigation.

Observe relationships through interactivity

After implementing the code, students can directly proceed with experimental observations. The plot in Fig. 4 shows the ground truth function f, the data points with noise $f(x_i) + \epsilon_i$, and the predicted function $\hat{f}(x)$. First, the possibility to manipulate every parameter (via the sliders above the plot) effectively communicates to students the impact of each parameter on a visual level. Second, through the manipulation of the parameters, students can analyze their relationship, e.g., how the regularization strength λ and the number of data points n_{samples} affect the test error for different degrees of the polynomial function $d_{\max}$ used for fitting. While this is typically a challenging task, as it requires exploring the interplay of four parameters, the corresponding plot provides students with direct feedback (and the instructor can further hint at certain relationships in subsequent questions).

Technical challenges and solutions

`scicode-widgets` depends on several external libraries that implement the widgets used as components in the applications. In addition to the package `ipywidgets`, that provides most of the standard widgets used for the applications, the coding environment relies on our package `widget-code-input`, providing the editable text box where students can input their code, where the function signature (and possibly function documentation in the format of a standard Python "docstring") are hardcoded and read-only (see its appearance in Figs. 1, 2, 3 and 4). While this widget might seem redundant, as notebook cells already allow students to edit code, it becomes essential when our widgets are combined with appmode (Ole Schuett, n.d.) or similar Jupyter extensions that run a complete notebook hiding all input cells, thus providing an "app-like" appearance to the notebook. Indeed, even if `scicode-widgets` reduces to a minimum the need for boilerplate code (see the top part of Fig. 1), hiding completely all code and showing only the resulting widgets further helps students in focusing only on the learning goal (see bottom of Fig. 1). Apart from some custom style sheets (CSS) code for styling, `scicode-widgets` does not include any JavaScript code and builds its widget applications from the building blocks provided by

DRAFT

its dependencies. In comparison to a two-language solution that would use JavaScript for the frontend combined with a Python interface for the code, this design choice has the advantage that the library is easier to maintain, as well as being accessible to a broader range of teaching instructors for code adaption, given Python's position as one of the most commonly used languages in an academic, education and research setting. In the following, we further discuss in more detail some of the technical challenges that we encountered during the development and the solutions we implemented.

Cued widgets

The exercise widget app is modular, allowing flexible initialization with only the components needed for a given use case. For example, the parameter box can be disabled in fixed experimental settings. The initialization process needs to handle several cases, which complicates the logic. To reduce the code redundancy, the logic for the system is handled in dedicated "cue widgets", that activate the appropriate visual cue when it observes a change in one of the traits (IPython Development Team, n.d.) in a list of widgets provided on initialization (i.e., in any of the other widgets that it observes for change). For the majority of the cueing, we use a `cue box` that wraps around an existing widget to highlight it around the edges. The behavioral change when a widget is cued is defined in a CSS file to allow for customization by the teaching instructor (e.g., color or line thickness). Furthermore, for resetting the cue, we implement a `reset cue` button that only resets the cue on a successful action. Moving this code logic into the individual components simplified handling of the logic in the exercise widget application and testing their functionality and correctness.

Saving mechanism

Since the target of our package is creating exercises for students, who then typically need to hand in their solution for grading, we need a mechanism to save the student solutions and transfer them to the teaching instructor. Jupyter environments can store widget states, including the HTML and JavaScript part of the widget. The execution of the code widget, however, requires communication with a Python kernel that runs the functions required for the data processing. We therefore need to re-execute the initialization of the widgets to recreate the corresponding Python objects, which would result in overwriting the static widget state, including the solutions provided by the students. To solve this issue, we implemented a custom saving mechanism for the exercise widgets, which stores both the code and the parameter values in a JavaScript Object Notation (JSON) file. The logic responsible for storing and loading the answers is in the exercise manager Python class, which retrieves the answer of an exercise widget.

Testing of the user interface

For an effective teaching experience, the robustness of the exercise application is essential. Indeed, the students are already required to consider errors in their understanding of the theory, the code, and interpretation of results. Adding failure of the widgets on top of these hurdles would affect very negatively the teaching experience. We therefore test the user interface of the exercise application extensively within a unit test framework, using the package `selenium` (Selenium Development Team, n.d.), which allows us to simulate realistic user interactions (e.g. typing on the keyboard or clicking with the mouse) via the web browser, and test the graphical output of the application.

Conclusion

In this work, we presented how an interactive widget application that combines a coding environment and visualization can be used to provide an engaging learning experience

DRAFT

for students. The creation of the widget application leaves enough room for flexibility to implement arbitrary logic for processing the student code, while taking care of the widget logic for the teaching instructor. This allows the teaching instructor to let arbitrary pre- and postprocessing of the student code happen "behind the scenes", so that students can fully focus on the main learning goal. We demonstrated the usage of `scicode-widgets` for teaching ridge regression, showing how the interactivity of the widgets enables seamless transitions between theory, code and analysis, resulting in interactions that would not occur as easily without them. `scicode-widgets` supports the most recent JupyterLab (Jupyter Development Team, n.d.) version (version 4 at the point of writing) and can be easily installed with a `pip` command. We therefore expect it to be straightforward to deploy and integrate into any local or cloud-based Jupyter deployment. These widgets have already been used to prepare a course at EPFL (MSE-305, Introduction to Atomic-Scale Modeling), and were refined based on the feedback from the students.

Data Availability

The code producing the `scicode-widgets` applications underlying each figure is available on Zenodo (Goscinski et al., n.d.). The notebooks used in the MSE-305 course are distributed in a dedicated repository (*Introduction to Atomistic Modeling, Course Notebooks*, n.d.).

Acknowledgements

We acknowledge financial support from the EPFL Open Science Fund via the OSSCAR project, from the EPFL DRIL fund titled "Jupyter web applications for quantum simulations", and from the NCCR MARVEL, a National Centre of Competence in Research, funded by the Swiss National Science Foundation (grant number 205602). We acknowledge CECAM for dedicated OSSCAR dissemination activities. A.G. and G.P. acknowledge funding by the SwissTwins project, funded by the Swiss State Secretariat for Education, Research and Innovation (SERI). M.C. and D.S. acknowledge funding from the European Research Council (ERC) under the research and innovation program (Grant Agreement No. 101001890-FIAMMA). We acknowledge useful discussions and feedback from Cécile Hardebolle and the students of the EPFL MSE-305 course – especially those of the academic year 2021-2022.

References

Bishop, C. M. (2006). *Pattern recognition and machine learning (information science and statistics)* (pp. 144–146). Springer-Verlag. https://doi.org/10.5555/1162264

Du, D., Baird, T. J., Bonella, S., & Pizzi, G. (2023). OSSCAR, an open platform for collaborative development of computational tools for education in science. *Computer Physics Communications*, *282*, 108546.

Du, D., Baird, T. J., Eimre, K., Bonella, S., & Pizzi, G. (2024). Jupyter widgets and extensions for education and research in computational physics and chemistry. *Computer Physics Communications*, *305*, 109353.

Goscinski, A., Baird, T. J., Du, D., Prado, J., Suman, D., Sodjargal, T.-E., Bonella, S., Pizzi, G., & Ceriotti, M. (n.d.). Scicode-widgets: Supplementary code for figures. In *Zenodo*. https://doi.org/10.5281/zenodo.15783260.

Granger, B. E., & Perez, F. (2021). Jupyter: Thinking and storytelling with code and data. *Computing in Science & Engineering*, *23*(2), 7–14. https://doi.org/10.1109/mcse.2021.3059263

DRAFT

Heradio, R., Torre, L. de la, & Dormido, S. (2016). Virtual and remote labs in control education: A survey. *Computers & Education*, *98*, 14–38. https://doi.org/10.1016/j.arcontrol.2016.08.001

Hu, D., Ahn, J. N., Lakatos, A., Bello, J., McTague, J., & Foley, J. J. I. (2021). *Integrating programming to reinforce quantum mechanical principles in physical chemistry* (Vol. 1387, pp. 89–105). American Chemical Society. https://doi.org/10.1021/bk-2021-1387.ch007

Introduction to atomistic modeling, course notebooks. (n.d.). GitHub repository: https://github.com/ceriottm/iam-notebooks.

IPython Development Team. (n.d.). *Traitlets: Lightweight eventful properties for python*. GitHub repository: https://github.com/ipython/traitlets.

Jupyter Development Team. (n.d.). *JupyterLab computational environment*. GitHub repository: https://github.com/jupyterlab/jupyterlab.

Ole Schuett. (n.d.). *Appmode: A jupyter extension that turns notebooks into web applications*. GitHub repository: https://github.com/oschuett/appmode.

Selenium Development Team. (n.d.). *Selenium browser automation project*. https://www.selenium.dev.

Stokes, G. Y. (2021). How faculty with minimal programming experience implemented jupyter notebooks in physical and general chemistry courses. In *Teaching programming across the chemistry curriculum* (pp. 13–27). American Chemical Society. https://doi.org/10.1021/bk-2021-1387.ch002

Sümmermann, M. L., Sommerhoff, D., & Rott, B. (2021). Mathematics in the digital age: The case of simulation-based proofs. *International Journal of Research in Undergraduate Mathematics Education*, *7*(3), 438–465. https://doi.org/10.1007/s40753-020-00125-6

Sutchenkov, A. A., & Tikhonov, A. I. (2020). Active investigation and publishing of calculation web based applications for studying process. *Journal of Physics: Conference Series*, *1691*(1), 012096. https://doi.org/10.1088/1742-6596/1691/1/012096

Thomas, K., Benjamin, R.-K., Fernando, P., Brian, G., Matthias, B., Jonathan, F., Kyle, K., Jessica, H., Jason, G., Sylvain, C., Paul, I., Damián, A., Safia, A., Carol, W., & Team, J. D. (2016). Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Tufino, E., Oss, S., & Alemani, M. (2025). Using jupyter notebooks to foster computational skills and professional practice in an introductory physics lab course. *Journal of Physics: Conference Series*, *2950*(1), 012022. https://doi.org/10.1088/1742-6596/2950/1/012022

Urcelay-Olabarria, I., Lazkoz, R., Urrestilla, J., Leonardo, A., & Igartua, J. M. (2017). Jupyter notebook as the physics experimental laboratory's logbook - first approach: *Proceedings of the 9th International Conference on Computer Supported Education*, 458–463. https://doi.org/10.5220/0006352104580463

Yakutovich, A. V., Eimre, K., Schütt, O., Talirz, L., Adorf, C. S., Andersen, C. W., Ditler, E., Du, D., Passerone, D., Smit, B., Marzari, N., Pizzi, G., & Pignedoli, C. A. (2021). AiiDAlab – an ecosystem for developing, executing, and sharing scientific workflows. *Computational Materials Science*, *188*, 110165. https://doi.org/10.1016/j.commatsci.2020.110165

DRAFT